

UNIVERSIDAD DE SEVILLA
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Grado en Ingeniería Informática - Tecnologías Informáticas
Curso 2026/2027

PAI-2

Grupo 33

Realizado por Daniel Enrique Molina Sanglade, Carlos Cobos Gordillo
y Ángela Castilla López

Índice

1. Resumen ejecutivo..... 3
2. Diseño del protocolo mTLS..... 4
3. Análisis de seguridad y benchmarks..... 5
3.1 Verificación de confidencialidad (trazas)..... 6
3.2 Prueba de 300 conexiones..... 6
4. Manual de Despliegue y Ejecución..... 6
5. Matriz de Trazabilidad de Requisitos..... 7
6. Anexo..... 8
6.1 Capturas de pantalla..... 8
6.2 Uso de IA..... 10

1. Resumen ejecutivo

La Universidad U-Secure ha crecido tecnológicamente los últimos años, y ahora permite a su personal acceder remotamente a recursos institucionales desde cualquier lugar utilizando dispositivos personales (BYOD). Sin embargo, conectarse desde redes externas no controladas (como Wi-Fi públicas o domésticas) expone la información a graves riesgos, como el robo de credenciales, la interceptación de datos o accesos no autorizados.

Para solucionarlo, el proyecto requiere diseñar una infraestructura VPN que establezca un canal de comunicación totalmente protegido entre los usuarios y la red corporativa. Esta solución debe garantizar la confidencialidad, integridad, autenticación mutua y trazabilidad, cumpliendo además estrictamente con normativas como el ENS, el RGPD y la LOPDGDD.

Para realizar este trabajo, se ha implementado una arquitectura basada en el protocolo mTLS 1.3 desarrollada en Python sobre sockets nativos, respaldada por una Autoridad de Certificación propia para la emisión y validación de certificados X.509v3. La solución se divide en dos partes: un modo sin certificado para el registro seguro de credenciales y expedición del certificado digital, y un modo mTLS con autenticación en dos capas (transporte y aplicación) que aplica Cipher Suites seguras, HMAC, nonces y la destrucción de claves efímeras tras el cierre de sesión.

El análisis mediante trazas de red y capturas de paquetes (.pcap) confirmó la confidencialidad e inteligibilidad del tráfico en tránsito. Además, la prueba de resistencia con la herramienta logins.py demostró la capacidad del sistema para gestionar 300 conexiones simultáneas con éxito, alcanzando un throughput medio de 11,93 conexiones/s y una latencia media RTT de handshake de 36,64 ms.

En definitiva, integrar mTLS 1.3 con una PKI dedicada permite blindar el modelo BYOD en entornos no confiables de forma eficaz. Además de garantizar un cifrado robusto y el cumplimiento de la normativa, el sistema responde con soltura a la carga operativa de la universidad, manteniendo una gran escalabilidad y un impacto mínimo en la latencia.

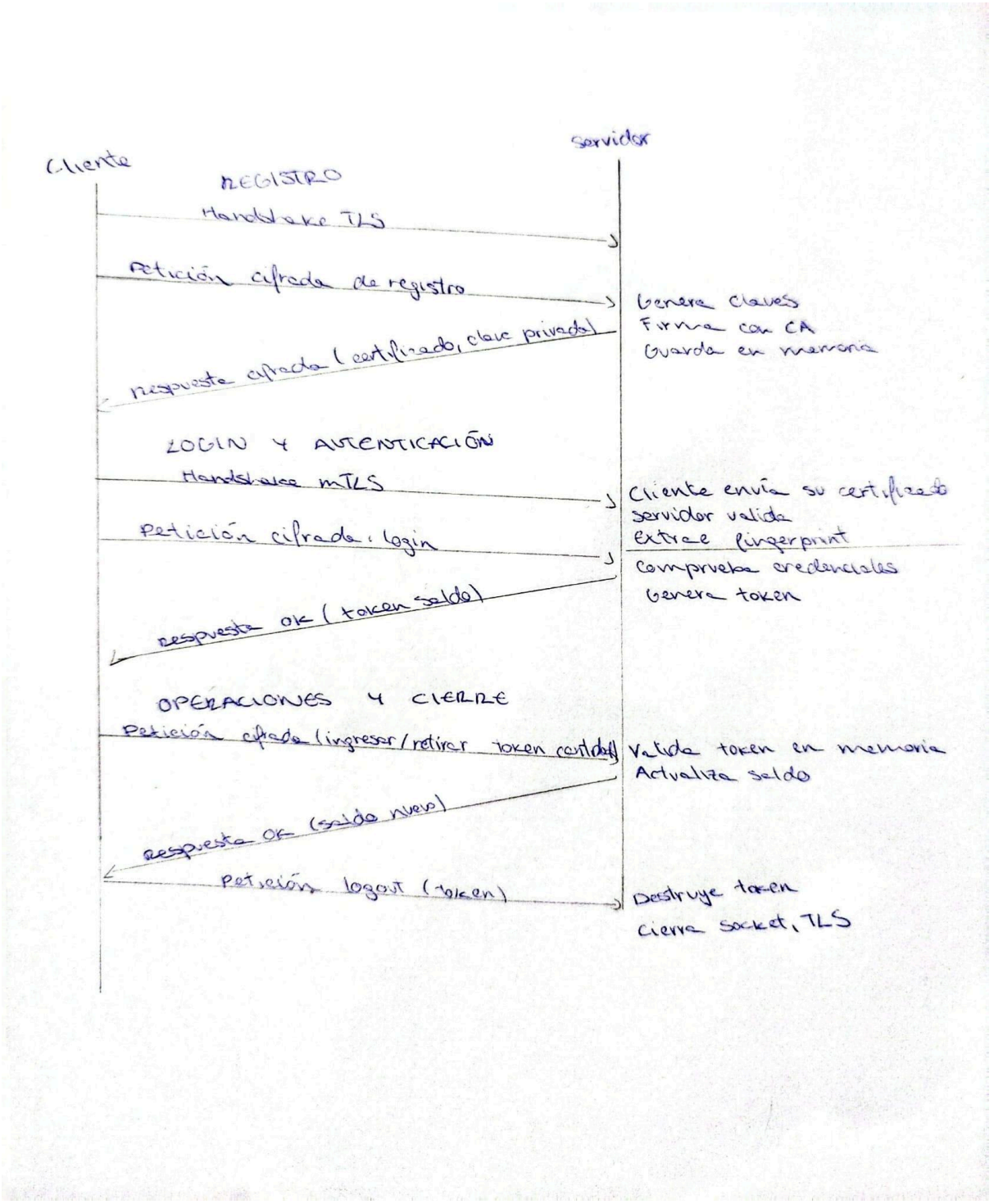
2. Diseño del protocolo mTLS

Similar a una conexión básica TCP, el programa da la opción de usar un modo con certificado (para utilizar la funcionalidad completa del programa) o sin certificado (restringido al registro y creación de nuevos certificados).

La conexión sin certificado atiende en primer lugar al registro de credenciales del usuario para posteriormente asegurar mediante encriptación la conexión y finalmente enviar clave y certificado del usuario a este, estando ya firmado por la entidad certificadora que ambos comparten.

La conexión con certificado requerirá de una primera autentificación de la validez del mismo y de ser correcta se permitirá el acceso al resto de funcionalidades mediante uso de mTLS 1.3 (que aplica siempre cipher suites seguras por encima de la encriptación/HMAC/nonces ya implementados)

A continuación se puede ver un diagrama de las conexiones que se tienen en este modelo cliente servidor (asumiendo registro y posteriormente login):



3. Análisis de seguridad y benchmarks

Para demostrar el correcto funcionamiento de las medidas de seguridad y robustez del sistema, hemos llevado a cabo pruebas de distintas modalidades.

3.1 Verificación de confidencialidad (trazas)

Para asegurar la confidencialidad del mensaje, hemos optado por explorar el intercambio de paquetes mediante trazas (Captura 1.1). Pudiendo comprobar que durante el registro, conexión y login las trazas son siempre inteligibles y utilizan cipher suites seguras (Traza anexada al documento para su mejor escrutinio).

3.2 Prueba de 300 conexiones

Con el propósito de asegurar la robustez del sistema, se ha utilizado el script "logins.py" para realizar una prueba con 300 bots que se conectarán al mismo tiempo al servidor y, tras un tiempo determinado, se desconectarán cerrando su sesión y liberando el semáforo (como se aprecia en la captura 1.2). Además de todo eso, se mide la latencia de las conexiones con el propósito de asegurar que no se pierde rendimiento y se compara con la latencia en conexiones no encapsuladas para comprobar la diferencia (Figura 1.3).

4. Manual de Despliegue y Ejecución

Para ejecutar el sistema hay que realizar los siguientes pasos:

En primera instancia, hay una serie de librerías que deben estar instaladas en los equipos de comunicación (algunas como cryptography, incluso en su misma versión), estas son: cryptography, mysql-connector-python y numpy. (El comando para su instalación será "pip install xxx", requiriendo por supuesto de alguna versión moderna de python >=3.15)

Es necesaria la instalación de OpenSSL light 3.4 para los certificados y la creación de los certificados y entidades certificadoras, no es necesario el uso de comandos externos dado que estas entidades ya aparecen en el repositorio.

Tras haber descargado estas librerías, hay que encender la base de datos, para ello usaremos XAMPP para encender un servidor SQL alojado en el puerto 3306 (por defecto), con usuario "root", sin contraseña y cuya IP es localhost (127.0.01). Dentro de este servidor usaremos el script sql "CreaciónDB" para la creación simple de los usuarios "admin" y "cliente". Como tercer paso, es necesario centrarnos en todos los scripts del proyecto y en concreto en "cliente.py", "servidor_enc.py" y "logins.py". Usaremos "servidor_enc.py" para iniciar el servidor en el dispositivo que haya iniciado la base de datos (basta con ejecutar el script), mientras que desde otro dispositivo usaremos "cliente.py" (o logins.py si queremos hacer distintas pruebas de rendimiento con 300 usuarios estableciendo conexión simultáneamente) poniendo la IP de nuestro servidor en el apartado del script indicado para ello (puede saberse usando el comando "ipconfig" si estás en windows), es importante que ambos dispositivos sean visibles entre ellos, por lo que deben estar conectados a la misma red. Por último, hay que asegurarse que en los archivos "seguridad.py" de las carpetas "cliente" y "server", la contraseña secreta es la misma. Si todas las comprobaciones son correctas, podemos ejecutar el script "cliente.py" que encontrará al servidor y podremos ejecutar nuestros comandos.

5. Matriz de Trazabilidad de Requisitos

Estado de todos los requisitos: cumplidos.

	Fichero	**Línea**
Objetivo 1	gestor_tls.py	1-23
Objetivo 2	servidor_enc.py gestor_tls.py	106 5
Objetivo 3	logins.py logins_old.py	1-112 1-75
Objetivo 4	Capturas de Wireshark (adjuntado aparte) y punto 3	-
Objetivo 5	Dado el uso de nonces y hmac estos ataques no surten efecto (mismo caso que en el PAI1)	-

Requisito funcional 1	servidor_enc.py	94-151
	gestor_tls.py cliente.py	 251
Requisito funcional 2	servidor_enc.py	219-222
Requisito funcional 4	servidor_enc.py	164
Requisito funcional 5	CreacionDB.sql	19-33
Requisito de Seguridad 1	servidor_enc.py	104-115
	cliente.py servidor_enc.py	49-72 61, 62
Requisito de Seguridad 2	gestor_tls.py	5, 15
	Implícito en la versión mínima requerida: gestor_tls.py	5
Requisito de Seguridad 3	servidor_enc.py	25, 251
	Capturas 2, 3 y 4 del anexo	-
Requisito de Seguridad 4	Capturas de Wireshark (adjuntado aparte)	-
	Capturas de Wireshark (adjuntado aparte)	-

6. Anexo

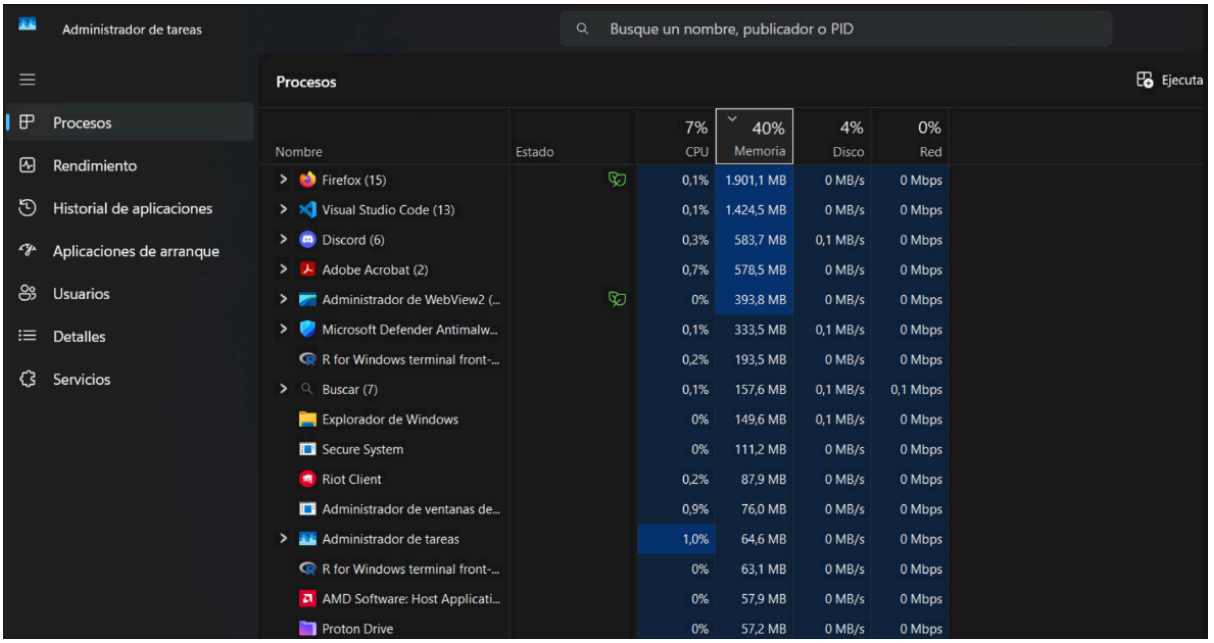
6.1 Capturas de pantalla

Captura 1:

```
================================================
 PRUEBA DE CARGA: 300 CONEXIONES TLS ANÓNIMAS SIMULTÁNEAS
================================================
[*] Conectando usuarios no registrados (Sin certificado cliente)...
[*] Todas las sesiones se mantendrán ABIERTAS hasta alcanzar 300 sockets activos.
[+] Estado conexiones: 300/300 procesadas...

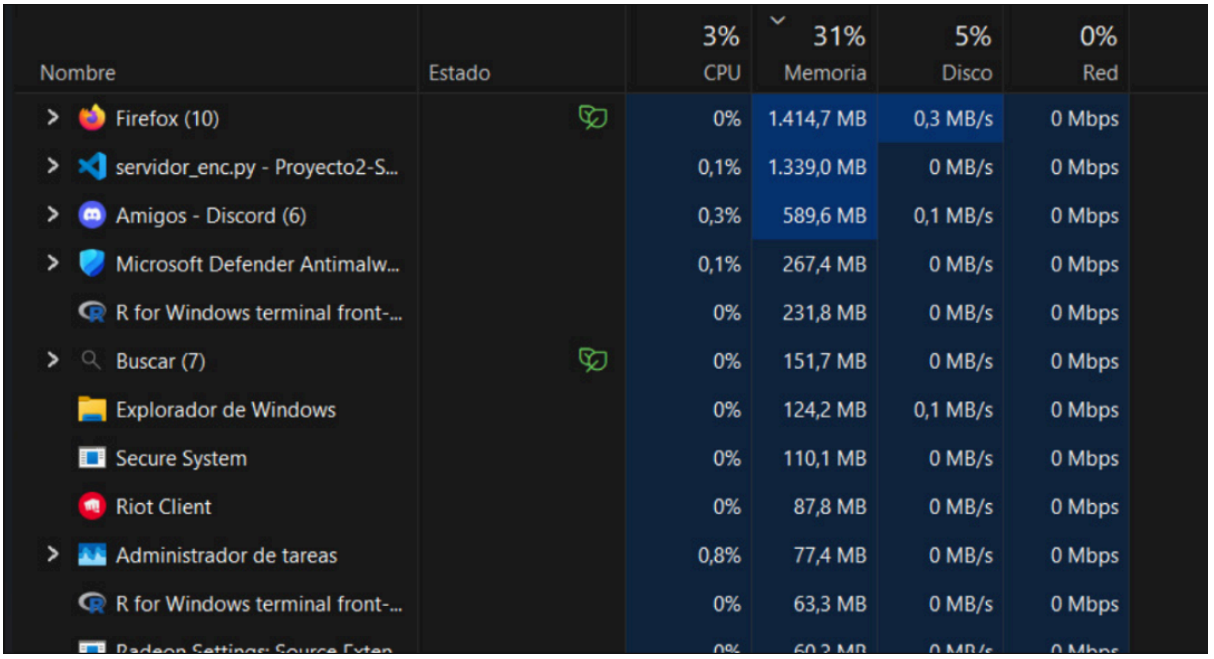
--- RESULTADOS DEL BENCHMARK ---
 Sesiones Conectadas Con Éxito:  300/300
 Conexiones Fallidas:            0/300
 Tiempo Total del Test:          25.144 s
 Throughput Medio:               11.93 conexiones/s
 Latencia Media RTT (Handshake): 36.64 ms
------------------------------------------------
```

Captura 2:



Consumo de CPU y memoria con 300 conexiones

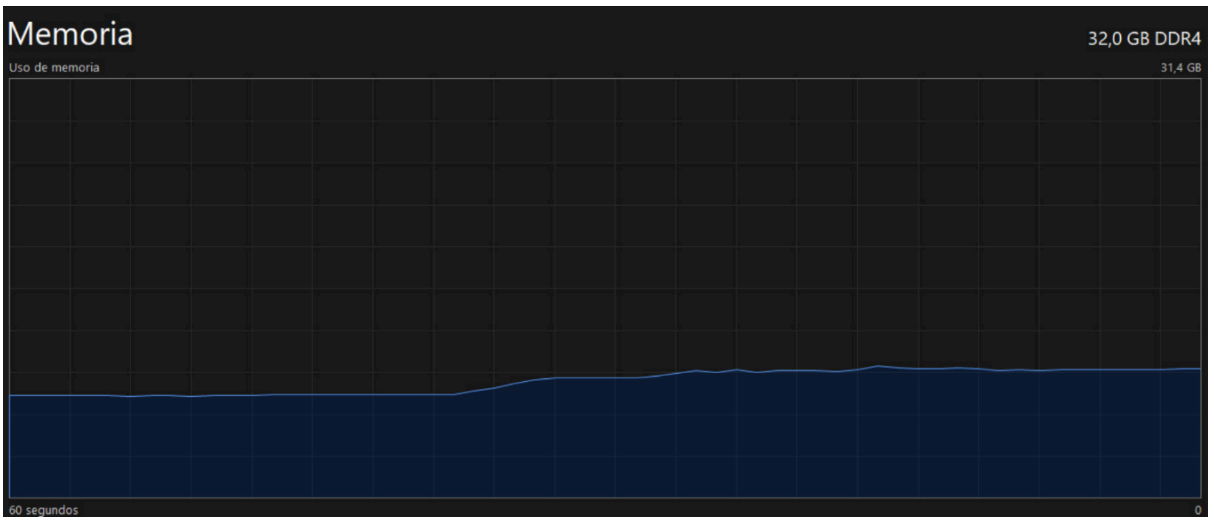
Captura 3:



Nombre	Estado	3% CPU	31% Memoria	5% Disco	0% Red
Firefox (10)		0%	1.414,7 MB	0,3 MB/s	0 Mbps
servidor_enc.py - Proyecto2-S...		0,1%	1.339,0 MB	0 MB/s	0 Mbps
Amigos - Discord (6)		0,3%	589,6 MB	0,1 MB/s	0 Mbps
Microsoft Defender Antimalw...		0,1%	267,4 MB	0 MB/s	0 Mbps
R for Windows terminal front-...		0%	231,8 MB	0 MB/s	0 Mbps
Buscar (7)		0%	151,7 MB	0 MB/s	0 Mbps
Explorador de Windows		0%	124,2 MB	0,1 MB/s	0 Mbps
Secure System		0%	110,1 MB	0 MB/s	0 Mbps
Riot Client		0%	87,8 MB	0 MB/s	0 Mbps
Administrador de tareas		0,8%	77,4 MB	0 MB/s	0 Mbps
R for Windows terminal front-...		0%	63,3 MB	0 MB/s	0 Mbps

Consumo de CPU y memoria con 0 conexiones

Captura 4:



Curva de comparación de la memoria entre 0 y 300 conexiones

6.2 Uso de IA

Utilización de la IA gemini para investigación y realización de partes del código, al igual que debugging:
https://share.gemini.google/gCYPIvHBz9ke